

ClaimFlow - Software Design Document

1. Project Overview

Project Name

ClaimFlow

Description

ClaimFlow is a two-sided web application that enables employees to submit expense reimbursement claims and allows finance reviewers to review, approve, reject, or return those claims for changes.

The system is designed around a controlled workflow process where all status transitions are enforced by the backend. Every workflow action is recorded in an audit trail to ensure transparency, accountability, and traceability.

The application demonstrates:

- Role-based authorization
- Backend-enforced workflow state machine
- Audit logging
- Server-side validation
- RESTful API design
- Automated testing
- PostgreSQL persistence

The primary focus of the project is enforcing valid status transitions and maintaining a complete audit history.

2. Business Roles

Applicant

Applicants are employees who submit expense reimbursement claims.

Can

- Create expense claims
- Save claims as drafts
- Edit draft claims
- Submit draft claims for review
- View their own claims
- View claim status history
- View audit logs

Cannot

- Approve claims
 - Reject claims
 - Return claims for changes
 - Review claims submitted by other users
 - Edit claims after submission
-

Reviewer

Reviewers are finance personnel responsible for evaluating expense claims.

Can

- View submitted claims
- Filter claims by status
- Start claim reviews
- Approve claims
- Reject claims
- Return claims for changes
- Add review comments
- View audit history

Cannot

- Modify applicant information
 - Edit submitted claim details
 - Submit claims on behalf of applicants
-

3. Expense Claim Fields

Each expense claim contains the following fields.

Field	Type	Required
title	string	Yes
category	enum	Yes
description	text	No
amount	decimal	Yes
attachmentUrl	string	No
status	enum	Yes

Expense Categories

- Travel
 - Fuel
 - Internet
 - Meals
 - Equipment
 - Other
-

4. Workflow State Machine

The workflow is the core business process of the application.

Statuses

```
DRAFT
SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
RETURNED_FOR_CHANGES
```

Valid Status Transitions

Applicant Actions

```
DRAFT → SUBMITTED

RETURNED_FOR_CHANGES → DRAFT
```

Reviewer Actions

```
SUBMITTED → UNDER_REVIEW

UNDER_REVIEW → APPROVED

UNDER_REVIEW → REJECTED

UNDER_REVIEW → RETURNED_FOR_CHANGES
```

Invalid Transitions

Examples of transitions that must be rejected:

DRAFT → APPROVED
SUBMITTED → APPROVED
REJECTED → DRAFT
APPROVED → DRAFT
Applicant → APPROVED
Applicant → REJECTED
Reviewer → Edit Claim Details

Invalid transitions return:

400 Bad Request

Unauthorized actions return:

403 Forbidden

5. Audit Trail

Every workflow transition generates an audit log record.

Audit logs provide a complete history of actions performed on a claim.

Audit Log Fields

- userId
 - applicationId
 - oldStatus
 - newStatus
 - comment
 - timestamp
-

Example Audit History

```
DRAFT → SUBMITTED  
  
SUBMITTED → UNDER_REVIEW  
  
UNDER_REVIEW → RETURNED_FOR_CHANGES  
  
RETURNED_FOR_CHANGES → DRAFT  
  
DRAFT → SUBMITTED  
  
UNDER_REVIEW → APPROVED
```

Audit history is displayed on the claim detail page.

Audit records are append-only and cannot be modified after creation.

6. Database Design

The application uses PostgreSQL as the primary data store.

users

Stores applicants and reviewers.

Column	Type
id	uuid
name	varchar
email	varchar
password_hash	varchar
role	enum
created_at	timestamp
updated_at	timestamp

applications

Stores expense reimbursement claims.

Column	Type
id	uuid
title	varchar
category	varchar
description	text
amount	decimal
attachment_url	varchar
status	enum
applicant_id	uuid
created_at	timestamp
updated_at	timestamp

application_audit_logs

Stores workflow history.

Column	Type
id	uuid
application_id	uuid
user_id	uuid
old_status	varchar
new_status	varchar
comment	text
created_at	timestamp

attachments

Stores uploaded receipt metadata.

Column	Type
id	uuid
application_id	uuid
file_name	varchar

Column	Type
file_url	varchar
mime_type	varchar
file_size	integer
created_at	timestamp

7. Authentication and Authorization

Authentication is implemented using JWT.

Authorization is enforced on every protected endpoint.

Seeded Accounts

Applicant

```
Email: applicant@test.com
Password: password123
```

Reviewer

```
Email: reviewer@test.com
Password: password123
```

Roles

```
APPLICANT

REVIEWER
```

Role checks are enforced on the backend and cannot be bypassed through direct API access.

8. API Endpoints

Authentication

POST /auth/login

Applicant Endpoints

GET /applications/my

GET /applications/:id

POST /applications

PATCH /applications/:id

POST /applications/:id/submit

Reviewer Endpoints

GET /reviewer/applications

GET /reviewer/applications/:id

POST /reviewer/applications/:id/start-review

POST /reviewer/applications/:id/approve

POST /reviewer/applications/:id/reject

POST /reviewer/applications/:id/return

9. Error Handling

All errors return structured JSON responses.

Validation Error

```
{
  "statusCode": 400,
  "message": "Amount must be greater than zero",
  "error": "Bad Request"
}
```

Invalid Workflow Transition

```
{
  "statusCode": 400,
  "message": "Invalid status transition",
  "error": "Bad Request"
}
```

Unauthorized Action

```
{
  "statusCode": 403,
  "message": "Forbidden resource",
  "error": "Forbidden"
}
```

Resource Not Found

```
{
  "statusCode": 404,
  "message": "Application not found",
  "error": "Not Found"
}
```

10. Testing Strategy

Testing focuses on workflow enforcement and authorization.

Unit Tests

State Machine Tests

- Valid transitions
 - Invalid transitions
 - Comment requirements
 - Status validation
 - Role restrictions
-

Authorization Tests

- Applicant cannot approve claims
 - Applicant cannot reject claims
 - Applicant cannot return claims
 - Reviewer cannot edit claim details
-

Integration Tests

- Unauthorized requests return 403
 - Invalid transitions return 400
 - Missing resources return 404
 - Successful transitions create audit records
 - Validation errors return structured responses
-

11. Technology Stack

Frontend

- React
 - TypeScript
 - React Hook Form
 - Zod
 - TanStack Query
 - Tailwind CSS
-

Backend

- NestJS
 - TypeScript
 - TypeORM
 - JWT Authentication
-

Database

- PostgreSQL
-

Testing

- Jest
 - Supertest
-

Containerization

- Docker
 - Docker Compose
-

12. Future Improvements

Potential enhancements include:

- Email notifications
 - Multi-level approval workflows
 - Approval delegation
 - Reviewer assignment
 - Receipt OCR
 - Expense policy validation
 - Activity feeds
 - Dashboard analytics
 - AWS S3 file storage
 - Real-time notifications
-

13. Architecture Principle

Business workflow rules are centralized in a dedicated service:

ApplicationStateMachineService

All workflow transitions must pass through this service.

Controllers remain thin and focus only on:

- Request validation
- Authentication
- Authorization

- Delegating business logic
-

Benefits

- Single source of truth
- Easier testing
- Better maintainability
- Clear separation of concerns
- Strong workflow enforcement
- Predictable business behavior

This architecture ensures that workflow rules remain consistent regardless of whether requests originate from the web application, automated tests, or direct API calls.